

Neural Leakage Model: Correlation Power Analysis with Profiled Leakage Model using Deep Neural Networks

Trevor Yap^{1,2}, Shivam Bhasin^{1,2} and Liu Zhang³

¹ Temasek Laboratories,

Nanyang Technological University, Singapore

² National integrated Centre For Evaluation,

Nanyang Technological University, Singapore

`{trevor.yap, sbhasin}@ntu.edu.sg`

³ School of Physical and Mathematical Sciences,

Nanyang Technological University, Singapore

`{liu.zhang}@ntu.edu.sg`³

Abstract. Side-channel analysis (SCA) exploits physical leakages such as power consumption or electromagnetic emissions to extract secret information from cryptographic implementations. Leakage model is the critical link between observable physical emanations (e.g., power consumption) and internal cryptographic states. When a leakage model fails to match the device’s underlying physical leakage, even powerful attacks like Correlation Power Analysis (CPA) are rendered ineffective. This fundamental challenge limits the success of traditional SCA, especially against noisy or masked targets. In this work, we introduce the Neural Leakage Model (NLM), a novel neural network architecture trained to learn and accurately characterize complex physical leakage to enhance CPA. Moving beyond NLM’s superior attack performance, we directly address the critical “black box” problem in deep learning-based side-channel analysis (DLSCA) by proposing a novel methodology to transform the trained NLM into a mathematically equivalent, closed-form polynomial expression. This provides interpretability, offering evaluators transparent insight into the precise leakage model the network has learned. We validate NLM’s performance across four diverse datasets spanning three platforms, from low-end microcontrollers to high-end multi-core ARM systems and dedicated FPGAs. Notably, NLM successfully recovers the secret key from the highly challenging and noisy CHES2025 dataset, representing the first known successful CPA attack on this benchmark. Furthermore, NLM demonstrates superior or comparable performance when extended to higher-order CPA against masking countermeasures. Our findings establish NLM as a powerful, generalizable, and interpretable approach to modern side-channel analysis.

Keywords: Side-channel, Neural Network, Interpretability, Correlation Power Analysis, Leakage Model

1 Introduction

With the rise of the Internet-of-things, side-channel analysis (SCA) has gained significant importance as a tool for evaluating the security of embedded devices [Lue21]. SCA recovers the secret data by exploiting physical properties such as timing delay [Koc96], power consumption [KJJ99], and electromagnetic emanation [AARR02]. It is typically applied in two settings: profiled and non-profiled. In the profiled setting, the attacker has access to an identical device to train a distinguisher. Conversely, in the non-profiled setting, the

attacker relies solely on the leakage from the target device, without access to a similar reference device.

The recovery of the secret data through SCA is critically dependent on the chosen leakage model. The overall effectiveness of the attack is thus determined by how well this model reflects the device’s actual physical leakage. These leakage models typically assume a linear correlation between the leakage and an intermediate variable, such as the Hamming weight or Hamming distance of the processed data. While these leakage models have been effective in many scenarios, they often fail to capture more complex, non-linear relationships present in real-world leakages, which can limit the success of traditional attacks. For example, Correlation Power Analysis (CPA) is a well-known attack that heavily relies on the chosen leakage model [BCO04]; however, it has been shown to fail in significantly noisier settings [BJW⁺25]. Therefore, there is a growing need to better characterize the leakage model.

Characterization of the leakage model not only enhances the effectiveness of the attack but also provides evaluators with critical insight into implementation vulnerabilities [KSS10, HKSS12]. This can help designers develop a secure implementation. For example, Kasper et al. [KSS10] used a stochastic leakage model and found that a specific bit of the sensitive variable contributes more significantly to the underlying leakage. They suggested that this behavior is linked to the underlying hardware components, particularly the multiplexer network. This insight enables designers to understand the flaws in the implementation and helps them develop appropriate countermeasures.

Despite these advances, classical approaches remain limited in how well they capture complex leakage behavior, which has led to increasing interest in DLSCA [BJW⁺25, WRA⁺24]. In recent years, DLSCA has gained attention for its ability to recover secret keys more effectively than classical attacks, both in the profiled setting [MPP16] and in the non-profiled scenario [Tim19, SKP⁺24]. Furthermore, DLSCA has been shown to successfully recover secret keys with minimal preprocessing, even on devices protected by hiding countermeasures such as desynchronization [CDP17]. In contrast, classical techniques typically require extensive preprocessing, such as trace realignment, for the attack to be effective. These preprocessing steps are often tedious and time-consuming. DLSCA has also been extended to other applications within the realm of SCA, like artificial traces generation [WCL⁺20, YJ24], collision attacks [SM23, WTPP24], and blind scenarios [RYJ⁺25]. Despite their success and numerous attempts to explain what the neural network is learning [SET23, YPB24], many DLSCA techniques suffer from a lack of interpretability. Nevertheless, deep neural networks are powerful tools. Its ability to learn complex, non-linear relationships makes it a promising candidate for characterizing leakage models.

Contribution. Our main contributions in this work are stated in the following:

- We propose the Neural Leakage Model (NLM), a novel neural network architecture inspired by the stochastic leakage paradigm. Built upon the widely used Multilayer Perceptron (MLP) architecture, NLM is fundamentally different from standard DLSCA architectures: it is explicitly designed to characterize the leakage by taking the sensitive intermediate value as input and modeling the output trace. This approach accurately captures the deterministic components of the side-channel signal, offering a powerful, flexible alternative to rigid, predefined classical models.
- To overcome the critical “black box” problem in DLSCA, we develop a technique using the Fourier Transform to convert *the trained NLM into an equivalent closed-form polynomial representation*. This innovation makes the learned leakage model fully interpretable by security evaluators, allowing for transparent analysis of the underlying physical leakage mechanisms.

- We validate the NLM across four diverse datasets originating from three distinct platforms, ranging from low-end 8-bit microcontrollers to high-end, multi-core ARM Cortex-A systems (>1 GHz) and dedicated FPGA hardware. Crucially, we achieve the first known successful CPA attack on the highly challenging, misaligned and noisy CHES2025 dataset, where all classical leakage models were reported to fail. Furthermore, we demonstrate NLM’s capability to learn both traditional (e.g., Hamming weight) and complex stochastic leakage models, and its successful extension to first-order masking defenses (ASCADf and ASCADr).

The source code can be accessed publicly at the following link.¹

Paper Organization. We first describe relevant prior work in Section 2, followed by the necessary background on SCA in Section 3. Section 4 presents the details of NLM and the process of converting it into a closed-form expression. Subsequently, the experimental results are presented in Section 5 and provide some discussion on the results in Section 6. Finally, we conclude the paper in Section 7 and outline potential directions for future work.

2 Related Works

Several leakage models were explored in the past to describe how physical leakages relate to internal data values. The simplest of these is the identity leakage model, which assumes a direct correspondence between leakage and intermediate values. Later, Hamming weight and Hamming distance models became widely adopted, as they assume that side-channel leakage correlates linearly with the number of active bits or bit transitions in intermediate values [ABDM00, BCO04]. Being defined a priori, these models are especially well-suited for non-profiled settings. Subsequently, to enhance attack performance, leakage models are profiled to more accurately characterize the underlying leakage. The stochastic leakage models proposed in [SLP05] are examples of such profiled models. These leakage models characterize the underlying leakage by performing linear regression on the individual bits of the sensitive variable. Thereafter, Jap et al. harness the power of machine learning by training Support Vector Regression (SVR) as the leakage model [JSB15]. They show that SVR could potentially learn a better leakage model that enhances the classical attack, like CPA or stochastic attack. However, interpreting which bits contribute to the leakage is challenging in SVR models, primarily because of the non-linear transformations introduced by the kernel function.

The use of deep neural networks to capture underlying leakage patterns in side-channel analysis is not a new concept [ZBC⁺23, BCG⁺25]. Zaid et al. [ZBC⁺23] first introduced the use of a Conditional Variational Autoencoder (cVAE) for stochastic attacks, enabling the model to compress traces while simultaneously learning the associated leakage behavior. Later, Boussam et al. [BCG⁺25] simplified the architecture to improve efficiency and reduce computational overhead. Although both approaches aim to learn the underlying leakage model, they rely exclusively on a linear layer for modeling the leakage model, which limits their expressiveness. Furthermore, their methods struggle to generalize to more complex scenarios, such as desynchronized traces or datasets protected by masking. In our work, our proposed NLM focuses only on learning the leakage model by incorporating non-linear layers. We will show that NLM can be applied to desynchronized datasets and first-order masked datasets.

Another research direction leverages deep neural networks to compress side-channel traces into embeddings for correlation analysis. Robyns et al. proposed Correlation Optimization by training a neural network that outputs a latent variable [RQL19]. The

¹Anonymized for review

deep neural network is trained such that it maximizes the correlation between the latent variable and the sensitive variable. This approach was later refined in multiple ways. For example, Hettwer et al. extended the work by considering the correlation based on the bits of the sensitive variable [HLF⁺21], while Chen et al. adapted the technique into a non-profiled scenario [CNK⁺22]. Nonetheless, these techniques do not offer any insight into the underlying leakage present in the traces. As a result, evaluators are unable to determine both the location and characteristics of the underlying leakage, even when the secret key is successfully recovered.

3 Preliminaries

3.1 Leakage Models

We denote a random variable through the use of capital letter X and a random vector by the bold capital letter $\mathbf{X}$. We denote the corresponding lowercase letters x and $\mathbf{x}$ to represent the realizations of X and $\mathbf{X}$, respectively. We let $x[i]$ stands for the i^{th} entry of a vector $\mathbf{x}$. When considering an element $x \in \mathbb{F}_{2^n}$, we use the notation $\mathbf{x} \in \mathbb{F}_2^n$ interchangeably. The $x \in \mathbb{F}_{2^n}$ refers to the field element, while $\mathbf{x} \in \mathbb{F}_2^n$ represents its corresponding vector representation. Furthermore, we define the vector $(\mathbf{x}_S)_{S \in \mathbb{F}_2^n} = (\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{2^n})$.

Consider a leaking device that computes a cryptographic primitive $f(P, k)$ where k is a subkey and P is a public random variable (e.g., plaintext or ciphertext). Furthermore, we define k^* as the correct key. A side-channel trace is defined as a random vector $\mathbf{T} = (T[0], \dots, T[D-1]) \in \mathbb{R}^D$ with D as the number of sample points (with $\mathbf{t} = (t[0], \dots, t[D-1])$ being its realization). Suppose $\{\ell_0, \dots, \ell_{s-1}\}$ are all indices where the sensitive information leaks, then for each sample point $j \in \{0, \dots, D-1\}$, the leakage value $T[j] \in \mathbb{R}$ is modeled as:

$$T[j] = \begin{cases} \Psi(f(P, k^*))[j] + \epsilon[j] & \text{if } j \in \{\ell_0, \dots, \ell_{s-1}\}, \\ \epsilon[j] & \text{otherwise,} \end{cases} \quad (1)$$

where $\Psi : \mathbb{F}_2^n \rightarrow \mathbb{R}$ is the underlying leakage model (i.e. deterministic part of the leakage) and $\epsilon \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ is the noise variable from the multivariate Gaussian Distribution with mean $\boldsymbol{\mu}$ and standard deviation Σ .

The goal of characterizing the leakage model is to find $\hat{\Psi}$, an accurate approximation of the true leakage model Ψ . In order to make the notation more concise, we set $Y_K = f(P, K)$ as the hypothetical sensitive variable and $\hat{Z}_K := \hat{\Psi}(f(P, K))$ to be the hypothetical sensitive variable with the chosen leakage model. In the following, we describe several classical leakage models commonly used for $\hat{\Psi}$.

Identity. The identity leakage model considers the whole sensitive variable to be leaking through the device, $\hat{\Psi} = \text{ID}$.

Hamming Weight & Hamming Distance. The Hamming weight leakage model (i.e. $\hat{\Psi} = \text{HW}$) was derived due to the assumption that the device is leaking based on the number of active bits in the intermediate sensitive variable [ABDM00]. The Hamming weight can be written as

$$\text{HW}(\mathbf{Y}) = \sum_{i=0}^n Y[i] \quad (2)$$

where $\mathbf{Y} = (Y[0], \dots, Y[n-1]) \in \mathbb{F}_2^n$.

Subsequently, Hamming distance (i.e. $\hat{\Psi} = \text{HD}$) is proposed as linearly with the number of bit transitions when loading sensitive variable into the register [BCO04]. The Hamming

distance is related to the Hamming weight by $\text{HD}(\mathbf{X}, \mathbf{Y}) = \text{HW}(\mathbf{Y} \oplus \mathbf{X})$ for $\mathbf{X}, \mathbf{Y} \in \mathbb{F}_2^n$. As the Hamming distance is equivalent to applying the Hamming weight to the XOR of two values, we will refer to both models under the unified term Hamming weight for simplicity.

Stochastic Leakage Model. In the previous leakage model, prior assumptions are made regarding the underlying leakage and can be used in a non-profiling setting. Stochastic leakage model is introduced as a profiled attack, an alternative to template attack [SLP05, GLP06, CK14]. It requires a profiling device to learn underlying leakage before using the trained leakage model as a distinguisher for the target device. The stochastic leakage models essentially characterize leakage using linear regression based on the bits of the sensitive variable. Since Ψ is a pseudo-boolean function, it can be viewed as a linear combination of monomial basis vectors $\mathbf{S} \in \mathbb{F}_2^n$ [Car20]. In this study, we consider only intermediate variables with $n = 8$ bits. Therefore, for an input $\mathbf{Y} = (Y[0], \dots, Y[7]) \in \mathbb{F}_2^8$, the $S256$ leakage model takes the form:

$$\hat{\Psi}_{S256}(\mathbf{Y})[j] = \sum_{\mathbf{S} \in \mathbb{F}_2^8} a_{\mathbf{S}} * \mathbf{Y}^{\mathbf{S}} \quad (3)$$

where $a_{\mathbf{S}} \in \mathbb{R}$ are the trainable parameters and $\mathbf{Y}^{\mathbf{S}} = \prod_{i=0}^n Y[i]^{S[i]}$ with $Y[i]^0 = 1$ and $Y[i]^1 = Y[i]$. This means that the true leakage Ψ can be approximated by a multivariate polynomial. The degree of $\hat{\Psi}_{S256}$ determined the maximal number of bits of interaction. Commonly, the degree is set beforehand, before the profiling phase. In addition to $S256$, we will also consider the $S9$ leakage model in this work. The $S9$ leakage model considers only the monomials of degree one. Hence, $S9$ can be written as:

$$\hat{\Psi}_{S9}(\mathbf{Y})[j] = a[0] + \sum_{i=1}^8 a[i] * Y[i-1] \quad (4)$$

where $\mathbf{a} = (a[0], \dots, a[8]) \in \mathbb{R}^9$ are the trainable parameters. Hence, HW leakage model can be viewed as a special case of the $S9$ leakage model where only the monomials of degree one have coefficients 1 and zero bias (i.e. $a[0] = 0, a[i] = 1$ for all $i \in \{1, \dots, 8\}$).

During profiling stage, a set of profiling traces and their corresponding sensitive variables are given, i.e. $\{(\mathbf{t}^{(0)}, \mathbf{y}^{(0)}), \dots, (\mathbf{t}^{(N_p-1)}, \mathbf{y}^{(N_p-1)})\}$. Given that $\mathcal{T} = (\mathbf{t}^{(0)}, \dots, \mathbf{t}^{(N_p-1)})^T$ to be the matrix that consists of all the N_p profiling traces collected. We consider $\mathcal{T}[j] = (\mathbf{t}^{(0)}[j], \dots, \mathbf{t}^{(N_p-1)}[j])$ to be the j^{th} sample points for all the traces. The goal is to find a set of trainable parameters $\mathbf{a}_j = (a_{j,\mathbf{S}})_{\mathbf{S} \in \mathbb{F}_2^8}$ (resp. $\mathbf{a}_j = (a_j[0], \dots, a_j[9])$ for $S9$) for each sample point $j \in \{0, \dots, D-1\}$ such that it minimizes the following Mean Square Error [Bis07]:

$$\min_{\mathbf{a}} \|\mathcal{T}[j] - \mathbf{a}_j \cdot \mathcal{Y}^T\|_2 \quad (5)$$

where $\mathcal{Y} = (\beta(\mathbf{y}^{(0)}), \dots, \beta(\mathbf{y}^{(N_p-1)})) \in \mathbb{F}_2^{N_p \times 2^8}$ where $\beta(\mathbf{Y}) = (\mathbf{Y}^{\mathbf{S}})_{\mathbf{S} \in \mathbb{F}_2^8}$ (resp. $\mathcal{Y} \in \mathbb{F}_2^{N_p \times 9}$ and $\beta(\mathbf{Y}) = (1, Y[0], \dots, Y[7])$ for $S9$). This can be solved via the ordinary least squares method [Bis07]. We note that Equation 5 applies to a single sample point. Typically, this is generalized to all sample points. We refer readers to [CK14] for a practical implementation of the ordinary least squares method applied across all sample points to obtain the stochastic leakage model.

3.2 Correlation Power Analysis

We will first recall the Pearson correlation as it is the foundation of CPA. The Pearson correlation of $\mathbf{x} = (x^{(0)}, \dots, x^{(N-1)})$ and $\mathbf{y} = (y^{(0)}, \dots, y^{(N-1)})$ is given as:

$$r(\mathbf{x}, \mathbf{y}) = \frac{\sum_{j=0}^{N-1} ((x^{(j)} - \bar{x})(y^{(j)} - \bar{y}))}{\sqrt{\sum_{j=1}^N (x^{(j)} - \bar{x})^2} \sqrt{\sum_{j=1}^N (y^{(j)} - \bar{y})^2}} \quad (6)$$

where $\bar{x}$ and $\bar{y}$ as the mean of $\mathbf{x}$ and $\mathbf{y}$ respectively. The Pearson correlation ranges from -1 to 1 (inclusive), and measures the degree of linearity between $\mathbf{x}$ and $\mathbf{y}$.

The CPA procedure can be summarized as follows:

1. Firstly, we compute the Pearson correlation between each sample point of all the traces and the hypothetical sensitive variable of a chosen leakage model $\hat{Z}_k$ for each key k .
2. Suppose ρ_k denotes the largest absolute correlation values among the sample points for key k . Sort the ρ_k in descending order and store them as $\boldsymbol{\rho} = [\rho_{k_{i_0}}, \dots, \rho_{k_{i_{|\mathcal{K}|-1}}}]$. The key corresponding to the score $\rho_{k_{i_0}}$ is the most likely candidate, and the key corresponding to the score $\rho_{k_{i_{|\mathcal{K}|}}}$ is the least likely candidate.
3. The index of guess vector $\boldsymbol{\rho}$ is called the rank of the key. The metric called guessing entropy GE_{corr} is defined as the average rank of the correct key k^* . If $GE_{corr} = 0$, when using N_a attack traces, the attack is considered successful. Due to the extensive computational nature of GE_{corr} , we run over 10 independent experiments. Let the metric $NTGE$ denote the minimum number of test traces required by the leakage model for GE_{corr} to reach and remain at 0. This indicates that the correct key consistently ranks first.

In most cases, the absolute value of the correlation coefficient is plotted against the sample points to visualize potential leakages within the traces. We note that CPA can be used in a non-profiled setting by computing the hypothetical sensitive variables with its leakage model $\hat{\mathbf{z}}_k = \hat{\Psi}(f(p, k))$, based on prior assumptions such as the identity or Hamming weight leakage models. It can also be applied in a profiled setting, where the leakage model $\hat{\Psi}$ is learned from training data to characterize the underlying leakage; for example, the stochastic leakage model is one of such cases.

Higher-order CPA. As CPA is inherently univariate, it becomes ineffective when the cryptographic primitive is protected using a countermeasure called masking [PR13]. It works by dividing the sensitive variable Y into multiple shares. More formally, a cryptographic primitive is said to have a masking order of d if the variable Y is split into $d + 1$ distinct shares, $Y = \gamma(m_1, m_2, \dots, m_{d+1})$ where the original value Y can be reconstructed using a general operator γ (for instance, in Boolean masking, γ corresponds to the XOR operation applied to all shares m_i for $i \in \{1, \dots, d + 1\}$).

In order to tackle the higher-order masking, higher-order CPA has been proposed [WW04, SPQ05, PRB10]. We begin by detailing how multivariate second-order CPA is performed. Firstly, every sample point within the traces are normalized, providing an identical off-sets. Next, each sample point is combine with every other sample point using combining functions giving us an extended traces. In this work, we will be using multiplication as the combining function. Other combining functions can be found in [PRB10]. Suppose we have D sample points for each trace, then we will have an extended trace with $D(D - 1)/2$ points. Finally, we simply apply the standard univariate CPA on these $D(D - 1)/2$ points. For the stochastic leakage model, we will profile on the extended trace of the profiling traces. Extending the second-order attack to higher-order CPA applies the combining function between each sample point and the existing combined points iteratively, up to the chosen order. Consequently, the attack's computational complexity increases exponentially with the order.

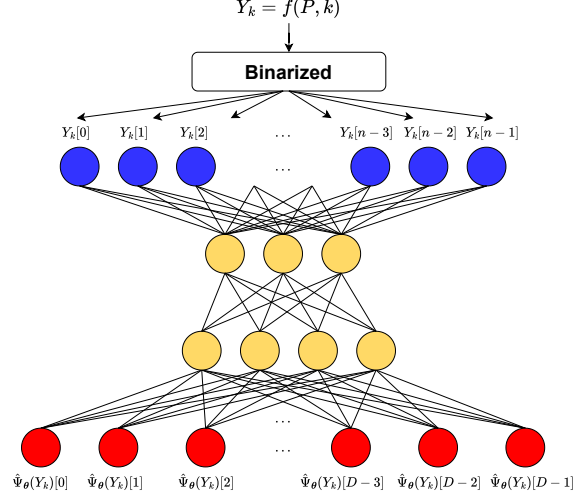


Figure 1: Neural Leakage Model Architecture with Binarization of Input.

4 Neural Leakage Model (NLM)

4.1 NLM Architecture and Loss Function

The NLM, denoted as Ψ_{θ} , is parameterized by trainable weights θ and instantiated either as a multilayer perceptron (MLP) or a convolutional neural network (CNN), both of which are standard architectures in modern machine learning. While CNN is known to obtain favorable results in SCA, our preliminary experiments with CNN-based NLMs yielded comparable performance to MLPs. In this study, we adopt the MLP variant of the NLM. Figure 1 gives an example of NLM architecture.

Given the intermediate sensitive variable $Y_K = f(P, K) \in \mathbb{F}_{2^n}$, the first step is to convert it into its binary representation consisting of n bits, i.e. $(Y_K[0], \dots, Y_K[n-1]) \in \mathbb{F}_2^n$. This representation conveys more detailed information to the neural network than the original element in $\mathbb{F}_{2^n}$ and facilitates the conversion of NLM into a closed-form expression that is understandable for the evaluator. We will denote the function $\text{bin}(x)$ to be the binarization function that maps $x \in \mathbb{F}_{2^n}$ to its corresponding n -bit binary vector representation $(x[0], \dots, x[n-1]) \in \mathbb{F}_2^n$. This binarized vector is then used as input to the NLM. The NLM's output, denoted by $\hat{\Psi}_{\theta}$, is expected to approximate the true leakage model Ψ . To support this approximation, the final layer of the NLM is designed as a simple linear layer with an output dimension of D . Formally, the NLM architecture can be represented as a neural network with l layers:

$$\hat{\Psi}_{\theta}(\mathbf{x}) = \mathbf{B}_{l-1}(\psi_{l-2} \circ \psi_{l-3} \circ \dots \circ \psi_0(\mathbf{x})) + \mathbf{w}_{l-1}. \quad (7)$$

Each layer is defined as $\psi_i(\mathbf{x}) = \varrho_i(\mathbf{B}_i \mathbf{x} + \mathbf{w}_i)$ where ϱ_i denotes the activation function (i.e., tahn, sigmoid, ReLU, SeLU, etc.). Here, $\mathbf{B}_i \in \mathbb{R}^{D_i \times D_{i-1}}$ and $\mathbf{w}_i \in \mathbb{R}^{D_i}$ where D_i represents the output dimension after the i layer. We set $D_0 = n$, corresponding to the number of input bits, and $D_{l-1} = D$, corresponding to the number of sample points. The full set of trainable parameters for the network is given by $\theta = \{\mathbf{B}_i, \mathbf{w}_i \mid i = 0, \dots, l-1\}$.

Next, to train the NLM, we minimize the following loss function:

$$\min_{\theta} \|\mathcal{T} - \Psi_{\theta}(Y_K)\|_2 \quad (8)$$

where $\mathcal{T}$ is the matrix containing all the N_p profiling traces. This objective corresponds to the Mean Squared Error (MSE), as in the stochastic leakage model. The model parameters θ are updated via gradient descent [GBC16] to minimize the MSE defined in Equation 8.

In contrast to the typical architectures used in DLSCA for profiling attacks, this architectural design takes the hypothetical sensitive variable as input and generates the corresponding ‘trace’ as output, more accurately, the deterministic part of the leakage.

CPA when using NLM. After training the NLM, each j^{th} component of NLM $\hat{\Psi}_\theta[j]$ is an approximation of the true leakage model $\Psi[j]$ at sample point $j \in \{0, \dots, D-1\}$. Therefore, to execute CPA, we set $\hat{Z}_k[j] = \hat{\Psi}_\theta[j](Y_k)$ as the chosen leakage model. It is worth noting that, like other leakage models, NLM can be applied in stochastic attacks or used with other distinguishers. However, in this work, we focus exclusively on CPA.

Higher-order CPA when using NLM. Similar to other leakage models, the use of CPA with NLM is inherently univariate. This means when the primitive is protected by masking, one needs to preprocess the traces by firstly normalizing every sample point, followed by the use of a combining function to obtain the extended traces. We will train the NLM and apply CPA on these extended traces in the same way as the stochastic leakage models.

While our approach allows us to apply CPA effectively, it provides limited insight into the internal behavior of the trained NLM. To gain a deeper understanding of what NLM has learned and to make its behavior interpretable, we next derive a closed-form representation of the NLM.

4.2 Close-Form Expression of the Neural Leakage Model

For a typical deep neural network, a closed-form expression generally does not exist. However, since the NLM takes only n binary inputs and produces real-valued outputs, each output component $\hat{\Psi}_\theta[j]$ for $j \in \{0, \dots, D-1\}$ is a psuedo-Boolean function, i.e. $\hat{\Psi}_\theta[j] : \mathbb{F}_2^n \rightarrow \mathbb{R}$. In this section, we are going to explain how to convert a pseudo-Boolean function into a closed-form expression.

We will first define the indicator function [O’D21]. The indicator function of $\mathbf{c} \in \mathbb{F}_2^n$, denoted by $\mathbb{1}_\mathbf{c} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ is the function defined as:

$$\begin{aligned} \mathbb{1}_\mathbf{c}(\mathbf{x}) &= \prod_{i=0}^{n-1} \frac{1 + (-1)^{c[i]+x[i]}}{2} \\ &= \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{c}, \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

Using the indicator function, any pseudo-Boolean function g can be represented as a polynomial representation (with respect to the Fourier basis):

$$g(\mathbf{x}) = \sum_{\mathbf{c} \in \mathbb{F}_2^n} g(\mathbf{c}) \mathbb{1}_\mathbf{c}(\mathbf{x}). \quad (9)$$

By expanding out the indicator function, we obtained the Fourier expansion of g [O’D21] which provides a unique polynomial representation in the Fourier basis.

Theorem 1. ([O’D21]) *Every psuedo-Boolean function $g : \mathbb{F}_2^n \rightarrow \mathbb{R}$ have an unique closed-form representation:*

$$g(\mathbf{x}) = \sum_{\mathbf{S} \in \mathbb{F}_2^n} b_\mathbf{S} * (-1)^{\sum_{i=0}^{n-1} x[i]*S[i]} \quad (10)$$

$$= \sum_{\mathbf{S} \in \mathbb{F}_2^n} b_\mathbf{S} * \phi_\mathbf{S}(\mathbf{x}) \quad (11)$$

where $b_{\mathbf{S}} \in \mathbb{R}$ is known as the Fourier coefficient of g on $\mathbf{S}$ and $\phi_{\mathbf{S}}(\mathbf{x}) = (-1)^{\mathbf{x} \cdot \mathbf{S}}$ with $\mathbf{x} \cdot \mathbf{S} = \sum_{i=0}^{n-1} x[i] * S[i]$.

The set of $\phi_{\mathbf{S}}$ forms a basis for the set of pseudo-Boolean functions ξ , which is also known as the Fourier basis [O'D21, GHTR17].² From the above theorem, this implies that for each component of NLM, $\hat{\Psi}_{\theta}$ can be represented as

$$\hat{\Psi}_{\theta}[j](\mathbf{y}) = \sum_{\mathbf{S} \in \mathbb{F}_2^n} b_{\mathbf{S}} * \phi_{\mathbf{S}}(\mathbf{y}) \quad (12)$$

As stated in [GHTR17], the Fourier basis allows for easier interpretation between the bits. Moreover, the basis is orthonormal, which implies that each basis vector is uncorrelated with the others. This allows us to understand the leakage model that NLM learns.

Rather than explicitly constructing the indicator function and expanding it to extract the coefficients, one can directly recover the coefficients $b_{\mathbf{S}}$ using Fourier analysis. To do this, we define an inner product on the space of pseudo-Boolean functions ξ as follows:

$$\langle f, g \rangle = \frac{1}{2^n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})g(\mathbf{x}) \quad (13)$$

for any pseudo-Boolean functions $f, g : \mathbb{F}_2^n \rightarrow \mathbb{R}$. The following result allows us to compute the Fourier coefficients directly:

Theorem 2. (Proposition 1.8 of [O'D21])

$$b_{\mathbf{S}} = \langle g, \phi_{\mathbf{S}} \rangle = \frac{1}{2^n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} g(\mathbf{x})\phi_{\mathbf{S}}(\mathbf{x}) \quad (14)$$

As a result, we can derive an explicit closed-form representation for the NLM.

Proposition 1. Each component of the NLM, denoted by $\hat{\Psi}_{\theta}[j]$ with $j \in \{0, \dots, D-1\}$, can be expressed as a Fourier expansion:

$$\hat{\Psi}_{\theta}[j](\mathbf{y}) = \sum_{\mathbf{S} \in \mathbb{F}_2^n} b_{\mathbf{S}} * \phi_{\mathbf{S}}(\mathbf{y}) = \sum_{\mathbf{S} \in \mathbb{F}_2^n} \langle \hat{\Psi}_{\theta}[j], \phi_{\mathbf{S}} \rangle * \phi_{\mathbf{S}}(\mathbf{y}) \quad (15)$$

where the Fourier coefficients are given by

$$b_{\mathbf{S}} = \langle \hat{\Psi}_{\theta}[j], \phi_{\mathbf{S}} \rangle = \frac{1}{2^n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} \hat{\Psi}_{\theta}[j](\mathbf{x}) * \phi_{\mathbf{S}}(\mathbf{x}) \quad (16)$$

Using Proposition 1, we develop an algorithm to derive the closed-form expression of the NLM, as presented in Algorithm 1, by computing all the coefficients $b_{\mathbf{S}}$ for each component of $\hat{\Psi}_{\theta}$. As a remark, Algorithm 1 can be readily adapted to any pseudo-Boolean function by replacing line 5 to the corresponding pseudo-Boolean function. Therefore, our technique can be extended to stochastic leakage models or SVR to obtain their corresponding Fourier basis formulation. Next, we examine the computational and memory complexity of Algorithm 1. Let M denote the computational cost of evaluating the model once. By precomputing and storing the model output over all 2^n input configurations, we incur a time complexity of $O(2^n * M)$ and a memory complexity of $O(2^n)$ (see line 3 to 6). To compute the complete set of Fourier coefficients $(b_{\mathbf{S},j})_{\mathbf{S} \in \mathbb{F}_2^n}$ for a single output component j , a time complexity of $O(n2^{2n})$ is needed (see lines 9 to 14). Since NLM have a total of D components, the overall time complexity becomes $O(n2^{2n} * D + 2^n * M)$ while

²[GHTR17] and [O'D21] differ by a scalar due to the difference in how they define the inner product. In this work, we will follow [O'D21].

Algorithm 1 Closed-Form Expression of NLM

Input: NLM $\hat{\Psi}_\theta$, number of bits n .
Output: Array $\mathcal{H}$ that stored the weights b_S at each S .

```

1: procedure CONVERTNLM( $\hat{\Psi}_\theta$ )
2:   Initilized an empty array  $\mathcal{W}$ .
3:   for  $s$  in  $\{0, 1, \dots, 2^n - 1\}$  do
4:     Binarize the inputs,  $S = \text{bin}(s)$ .
5:     Store NLM output at  $s$  in the array  $\mathcal{W}[s] = \hat{\Psi}_\theta(S)$ .
6:   end for
7:   Initialize the empty array  $\mathcal{H}$ .
8:   for component  $j$  in  $\{0, 1, \dots, D - 1\}$  do
9:     for  $s$  in  $\{0, 1, \dots, 2^n - 1\}$  do
10:       $Coeff = 0$ .
11:      for  $x$  in  $\{0, 1, \dots, 2^n - 1\}$  do ▷ Compute  $b_S = \langle \hat{\Psi}_\theta[j], \phi_S \rangle$ 
12:        Binarize  $x = \text{bin}(x)$  and  $S = \text{bin}(s)$ 
13:         $Coeff = Coeff + \mathcal{W}[x] * (-1)^{x \cdot S}$ 
14:      end for
15:       $\mathcal{H}[s, j] = Coeff$ 
16:    end for
17:  end for
18:  return array  $\mathcal{H}$ 
19: end procedure

```

Table 1: Elapse Time for running Algorithm 1.

	AES_HD	CHES2025	ASCADf	ASCADr
Time	≈ 7 mins	≈ 40 mins	≈ 5 mins	≈ 5 mins
No. of Sample Points	1250	7000	780	780

the memory complexity remains $O(2^n)$ (see line 8). Therefore, if n is sufficiently small, i.e. $n = 8$, we can feasibly enumerate all possible inputs to the NLM and compute its complete Fourier representation. In fact, Algorithm 1 runs quite efficiently on the four datasets tested. Table 1 reports the elapsed time required to execute the algorithm on each dataset. Since $n = 8$ for all four datasets, the time is mainly dependent on the number of components/sample points, D , and the computational complexity of NLM, M .

Overall, this yields an explicit closed-form expression for the trained NLM. The resulting closed-form representation reveals which input bits interact, offering insight into the structure of the leakage model learned by the network. If the CPA succeeds, it indicates that the network has successfully captured the underlying leakage model. The resulting closed-form expression then allows evaluators/designers to identify which interactions between input bits contribute to the leakage, providing valuable insights that can guide modifications to the underlying design or implementation.

5 Experiments

5.1 Hyperparameter search for NLM

Since the hyperparameters of the neural networks affect performance, we randomly sampled different hyperparameters from the ranges listed in Table 2. For each dataset, we trained 100 different MLP models as our NLM. Next, we applied CPA using the trained NLMs. Then, we visually examined the CPA graphs (correlation vs. sample points) of the NLMs. We then randomly selected one of the NLMs that successfully recovered the key for further analysis. Based on our initial tuning, we use 500 epochs to train our NLM for AES_HD and CHES2025 while using 200 epochs for ASCADf and ASCADr instead.

Table 2: Hyperparameter search space.

MLP	
Hyperparameter	Options
Number of Dense Layers	1 to 8 in a step of 1
Neurons per layer	10, 20, 50, 100, 200, 300, 400, 500
Batch size	100 to 1000 in a step of 100
Activation function	<i>ReLU</i> , <i>SeLU</i> , <i>ELU</i> or <i>tanh</i>
Optimizer	Adam or RMSprop
Learning Rate	$1e-3$, $1e-4$, $5e-4$, $1e-5$, $5e-5$
Weight Initializer	Random Uniform or Glorot Uniform or He Uniform

5.2 AES_HD

Dataset. The AES_HD is a dataset collected from an unprotected hardware implementation of AES executed on an FPGA using a round-based architecture. It contains of 500000 traces, with each consisting of 1250 sample points. The sensitive variable we are targeting will be the last round leakage:

$$Y_{k^*} = Sbox_{AES}^{-1}(CT_{15} \oplus k_{15}^*) \oplus CT_{11} \quad (17)$$

with CT_i denotes the i^{th} ciphertext byte and k_{15}^* denotes the 15th byte of the last round key. When profiling the leakage model, specifically when training the NLM or solving the ordinary least squares model for stochastic leakage models, we used 45k profiling traces. We then used 5000 attack traces for our CPA. For the identity and Hamming weight leakage models, we considered only the same 5000 attack traces, as these models do not require any profiling stage.

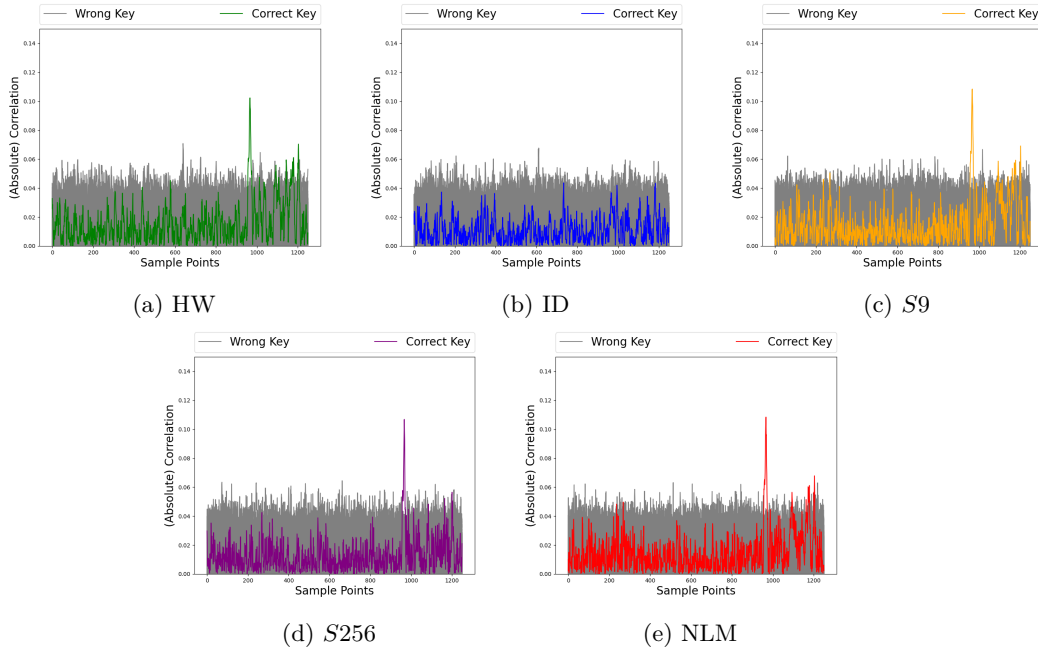


Figure 2: CPA on AES_HD dataset with classical leakage models and NLM

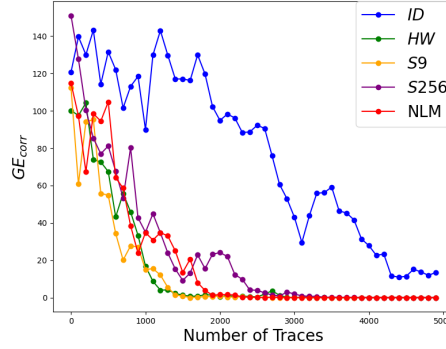
Experiment Results. Firstly, we run CPA with the different classical leakage models and our proposed NLM. Figure 2 illustrates the correlation versus sample points for all

considered leakage models. In Figure 3b, we also plot the GE_{corr} versus the number of attack traces, with the corresponding $NTGE$ values summarized with the corresponding $NTGE$ in Table 3a with $-$ indicating that the corresponding leakage model failed to recover the secret key within the available traces. The GE_{corr} and $NTGE$ are computed over a range of 100 to 5000 attack traces, evaluated at intervals of 100 traces.

We observe that the ID leakage model struggles to recover the secret key, resulting in a high guessing entropy. In contrast, the HW, S9, S256, and NLM models are able to successfully recover the secret key. Among all models, the S9 leakage model performs best, achieving an $NTGE$ of 2700. The proposed NLM also demonstrates strong performance, requiring comparable 2900 attack traces to reach $GE_{corr} = 0$. This indicates that the NLM is competitive with classical leakage models such as ID, HW, S9, and S256.

	ID	HW	S9	S256	NLM
$NTGE$	$-$	3500	2700	3900	2900

(a) $NTGE$ for various leakage models.



(b) GE_{corr} with all the various leakage models.

Figure 3: GE Result for AES_HD (interval of 100 traces).

Next, we aim to understand the bit interactions of the sensitive variable learned by the NLM. To do this, we convert the NLM into a closed-form expression as stated in Section 4.2. Components of the NLM that achieve the top correlations for the correct key tend to have similar closed-form expressions. Therefore, we will only showcase the component with the highest correlation w.r.t the correct key, i.e. the component 966 for AES_HD. Namely, we will provide the closed-form expression of $\hat{\Psi}_{\theta}[966](\mathbf{y})$.

Due to the length of the full expression, we present only the ten largest absolute coefficients of $\hat{\Psi}_{\theta}[966](\mathbf{y})$ and refer readers to our weblink for the full expression. Therefore, truncated expression $\hat{\Psi}_{\theta}[966](\mathbf{y})$ is given by:

$$\begin{aligned}
\hat{\Psi}_{\theta}[966](\mathbf{y}) = & 0.04329662 * \phi_1(\mathbf{y}) + 0.03392301 * \phi_2(\mathbf{y}) + 0.03675886 * \phi_4(\mathbf{y}) \\
& + 0.03537344 * \phi_8(\mathbf{y}) - 0.00410855 * \phi_9(\mathbf{y}) + 0.04877001 * \phi_{16}(\mathbf{y}) \\
& + 0.03185472 * \phi_{32}(\mathbf{y}) + 0.00413907 * \phi_{48}(\mathbf{y}) + 0.03610843 * \phi_{64}(\mathbf{y}) \\
& + 0.02452886 * \phi_{128}(\mathbf{y}) + \dots
\end{aligned} \tag{18}$$

Figure 4a shows the values of the absolute coefficients $|b_{\mathbf{S}}|$ for each $\mathbf{S}$ of $\hat{\Psi}_{\theta}[966](\mathbf{y})$. We consider the absolute values of the coefficients to identify which bit interactions contribute most significantly to the leakage model. The x-axis represents the value of $\mathbf{S}$ interpreted as an element of the finite field $\mathbb{F}_{2^n}$. For example, if the x-axis value is 9, this corresponds to $\mathbf{S} = (0, 0, 0, 0, 1, 0, 0, 1)$ which reflects interactions among the bits $Y_k[4]$ and $Y_k[7]$ via

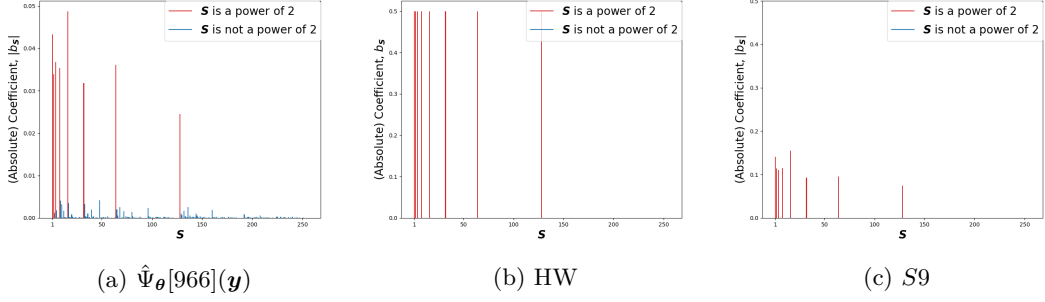


Figure 4: (Absolute) Coefficients for closed-form expression of the NLM closed-form expression of the NLM at the component with the highest correlation (Component 966), $S9$, and HW on AES_HD .

the Fourier basis ϕ_S . In the figure, values of S that are of power 2 are highlighted in red, while all other values are shown in blue. The red points correspond to the Fourier basis of the form $(-1)^{Y_k[i]}$, indicating leakage contributions only from individual bits of the sensitive variable. In contrast, the blue points represent the Fourier basis of multiple bits, capturing leakage resulting from bit interactions. We ignore $S = 0$ in our figure as it is a constant and does not show any bit interactions. We note that the HW can be rewritten in terms of the Fourier basis as:

$$HW(\mathbf{y}) = \frac{n}{2} - \frac{1}{2} \sum_{i=0}^{n-1} \phi_{2^i}(\mathbf{y}). \quad (19)$$

Hence, we plot the same graph for the HW leakage model in Figure 4b. In fact, the S with the highest absolute coefficient $|b_S|$ are all the power of 2 (i.e., $S = 1, 4, 8, 16, 32, 64, 128$) with $S = 16$ with the highest value of $|b_S|$. On the other hand, for the best distinguisher $S9$, the maximum correlation of the correct key obtained is also at sample point 966. We convert Equation 4 at sample point 966 into an equation in terms of the Fourier basis via a slight modification of Algorithm 1 (by changing the NLM output to $S9$ output in line 5) and obtained the following:

$$\begin{aligned} \hat{\Psi}_{S9}(\mathbf{Y})[966] = & 70.5 + 0.14103619 * \phi_1(\mathbf{y}) + 0.114124373 * \phi_2(\mathbf{y}) \\ & + 0.110609985 * \phi_4(\mathbf{y}) + 0.115215425 * \phi_8(\mathbf{y}) - 0.154638519 * \phi_{16}(\mathbf{y}) \\ & + 0.093047467 * \phi_{32}(\mathbf{y}) + 0.095232622 * \phi_{64}(\mathbf{y}) + 0.074453545 * \phi_{128}(\mathbf{y}). \end{aligned} \quad (20)$$

This shows that $S9$ is just a weighted version of the HW leakage model even in terms of the Fourier basis (plotted in Figure 4c). Consequently, the NLM appears to have learned a leakage model closely resembling both HW and $S9$, while also capturing additional bit-interaction terms (e.g., $S = 9$) that are absent from these classical models. This explains why the NLM achieves performance comparable to that of the traditional leakage model.

5.3 CHES2025

Dataset. The CHES2025 public dataset, prepared for the CHES 2025 challenge [BBJ⁺25]. This dataset consists of side-channel traces collected from a Raspberry Pi 4B board featuring a Broadcom BCM2711 System-on-Chip (SoC). The device is powered by a quad-core, out-of-order ARM Cortex-A72 processor (ARMv8, 64-bit), running at up to 1.8 GHz. It operates on the Bullseye version of Raspberry Pi OS (based on Debian Linux). These

processors are modern, high-end and can be found in everyday gadgets like smartphones and wearables products. The traces were captured during the execution of an unprotected, C-based implementation of AES-128, compiled without any compiler optimizations. Due to the out-of-order execution of the processor, which results in noisy and inherently desynchronized power traces. Figure 5 presents an overlay of 10 traces from the CHES2025 dataset used.

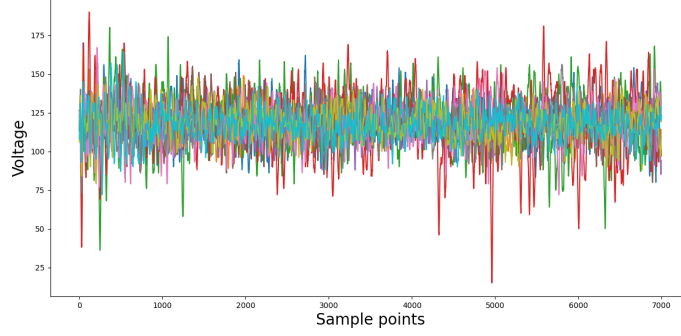


Figure 5: Plot of 10 traces of CHES2025 dataset

In our experiments, we focus exclusively on the **fixed key** with the target on the first byte of the SubBytes Operation in the first round:

$$Y_k = Sbox_{AES}(pt_0 \oplus k_0^*) \quad (21)$$

where pt_0 is the first plaintext byte and k_0^* denotes the first byte of the first round key. We use 45000 profiling traces for training and 10000 attack traces for conducting the experiments. The same set of attack traces is also employed for both the identity and Hamming weight leakage models for CPA.

Experimental Results. Similar to the previous dataset, we begin by performing CPA using classical leakage models (see Figure 6a to 6d). However, none of the classical leakage models were able to successfully recover the secret key either. A similar observation was reported in [BJW⁺25] where their traces are obtained in a similar setting. This behavior is likely attributable to the high levels of noise and desynchronization present in the traces caused by the out-of-order execution of the processor, which resulted in the underlying leakage being more complex and more difficult to characterize.

Despite that, using our proposed NLM, we managed to recover the secret key. Figure 6e shows that the leakages are mostly leaking around the 750th sample point. To the best of our knowledge, this is the first time the precise locations of the leakages in the CHES2025 dataset have been identified. Furthermore, we plot the GE_{corr} for the various leakage models in Figure 7b. We observe that only NLM manages to obtain a $GE_{corr} = 0$ with $NTGE$ of 7600 attack traces (see Table 7a). According to the leaderboard in [BBJ⁺25], when sorted by $NTGE$ on the public dataset only, our NLM achieves the 12th best score. This shows that NLM is capable of accurately modeling the leakage, reducing the traces needed for key recovery, and demonstrating NLM’s practical advantage in real-world scenarios.

Thereafter, we look into the closed-form expression of the NLM. We convert NLM into its closed-form representation using Algorithm 1 to obtain the coefficient b_S for each S . We shall focus on $\hat{\Psi}_\theta[759](y)$ as it attains the highest correlation value with respect to the correct key. As before, we shall present only the ten largest absolute coefficients of

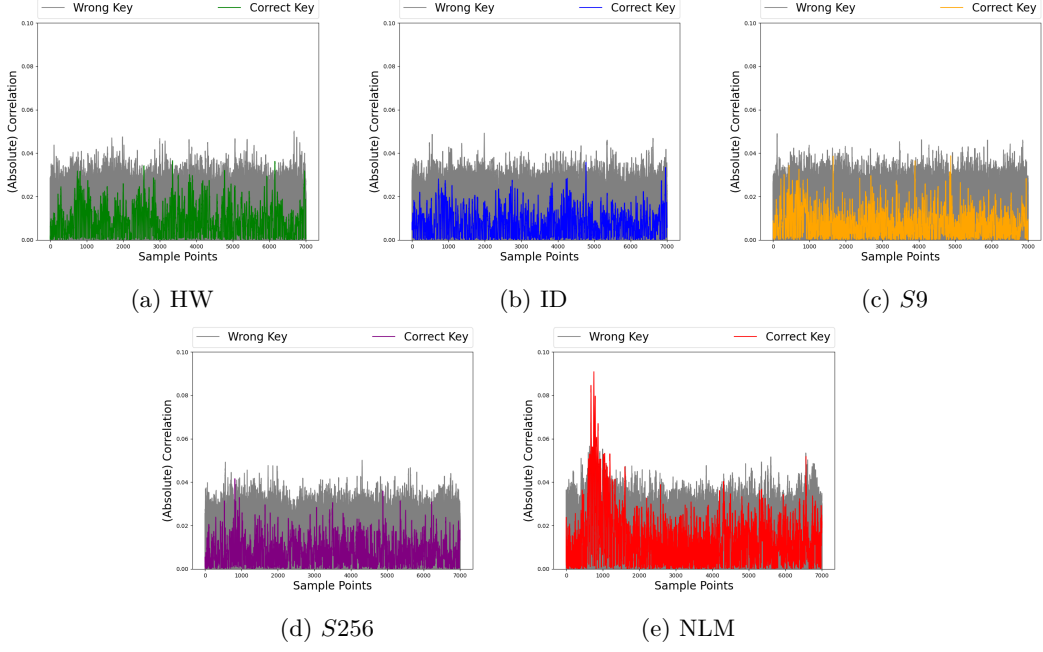


Figure 6: CPA on CHES2025 dataset with classical leakage models and NLM

	ID	HW	S9	S256	NLM
<i>NTGE</i>	—	—	—	—	7600

(a) NTGE for various leakage models.

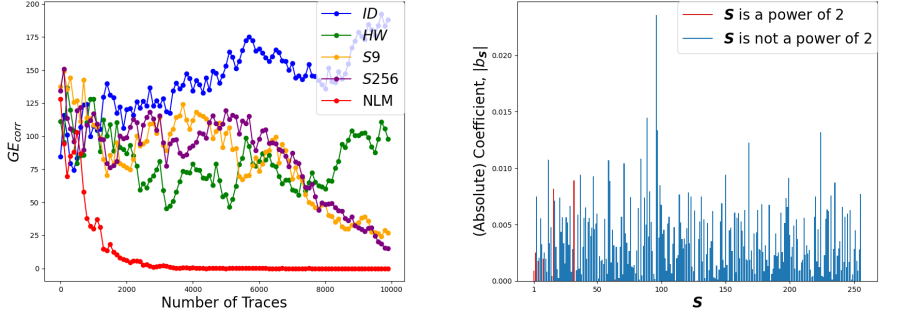
(b) GE_{corr} with all the various leakage models.(c) Absolute Coefficients for $\hat{\Psi}_{\theta}[759](\mathbf{y})$

Figure 7: GE Result for CHES2025 dataset (interval of 100 traces) and the absolute coefficients for closed-form expression of the NLM at the sample point with the highest correlation on CHES2025 (Component 759).

$\hat{\Psi}_{\theta}[759](\mathbf{y})$ and refer the reader to our source code for the full expression. The truncated

expression of $\hat{\Psi}_\theta[759](\mathbf{y})$ is given by:

$$\begin{aligned} \hat{\Psi}_\theta[759](\mathbf{y}) = & 0.01074025 * \phi_{12}(\mathbf{y}) - 0.01069283 * \phi_{59}(\mathbf{y}) - 0.01041968 * \phi_{71}(\mathbf{y}) \\ & + 0.01083409 * \phi_{84}(\mathbf{y}) - 0.01445224 * \phi_{89}(\mathbf{y}) - 0.02358096 * \phi_{96}(\mathbf{y}) \\ & - 0.01336265 * \phi_{97}(\mathbf{y}) - 0.00943004 * \phi_{150}(\mathbf{y}) - 0.01224716 * \phi_{168}(\mathbf{y}) \\ & - 0.01320333 * \phi_{224}(\mathbf{y}) + \dots \end{aligned} \quad (22)$$

Next, Figure 7c shows all the absolute values of $b_{\mathbf{S}}$ for the component 759. Unlike AES_HD, the underlying leakage here is more complex due to the processor’s out-of-order execution environment. As a result, we observe that the NLM models the deterministic part of the leakage as a highly nonlinear function, involving numerous bit interactions. This is evident from the fact that the coefficients are mostly not a power of 2 (more blue than red lines in Figure 7c). In fact, the highest absolute coefficient $|b_{\mathbf{S}}|$ is when $\mathbf{S}$ is 96, i.e. $\mathbf{S} = (0, 1, 1, 0, 0, 0, 0, 0)$. This suggests that the leakage is heavily influenced by bits $Y_k[1]$ and $Y_k[2]$ at this sample point. A natural question arises as to why S_{256} , despite its capacity to model nonlinear functions, fails to recover the secret key. This phenomenon is examined in greater detail in Section 6.

5.4 First-Order Masking CPA

In this section, we investigate whether we can recover the secret key using second-order CPA with NLM on datasets protected by first-order masking. Specifically, we will examine the ASCAD dataset.

Datasets. The ASCAD dataset is a first-order masked AES implementation on an 8-bit AVR microcontroller (ATMega8515) [BPS⁺20]. We target the third byte of the first round of the AES substitution box, which is protected by first-order masking. We denote the sensitive variable that we are targeting as

$$Y_k = Sbox(pt \oplus k^*) \quad (23)$$

where pt is the third plaintext byte and k^* is the third byte of the first round key. The dataset contains two versions known as ASCADf and ASCADr. ASCADf comprises traces generated using a fixed key, whereas ASCADr contains profiling traces collected under the random key setting and attack traces acquired using a fixed key. For both datasets, we use 45000 traces for profiling and 10000 attack traces. For both datasets, we consider truncated traces: ASCADf traces are truncated to 700 sample points, while ASCADr traces are truncated to 1400 sample points. When applying higher-order CPA, we consider 20 sample points from each share, and then combine them pairwise (by multiplication) to produce an extended trace with 780 sample points. The leakage on this device is known to closely follow the HW model, as reported in previous works [BPS⁺20].

5.4.1 Experimental Results for ASCADf.

We run CPA on the extended traces (after normalizing and using the combining function) with various leakage models that we considered. We provide the correlation versus sample point for all the leakage models tested in Figure 8.

Next, we present the plot of GE_{corr} versus the number of traces in Figure 9b along with the corresponding $NTGE$ summarized in Table 9a. As before, we confirm that ID leakage could not retrieve the secret key. On the other hand, using NLM, HW, S_9 , and S_{256} managed to recover the secret key. In fact, S_9 recovers the secret key with the least number of attack traces, using 3200 traces, followed by NLM with 4000 attack traces. This shows that NLM can indeed perform comparably to the classical leakage models.

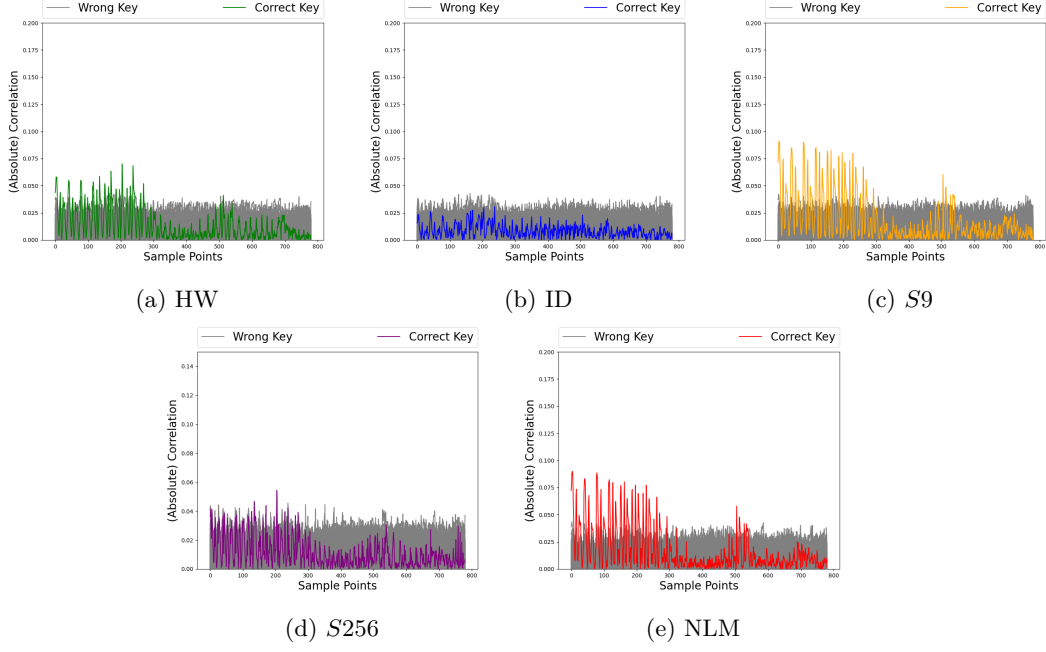


Figure 8: CPA on ASCADf datasets with classical leakage models and NLM

	ID	HW	S9	S256	NLM
<i>NTGE</i>	—	6500	3200	9000	4000

(a) NTGE for various leakage models.

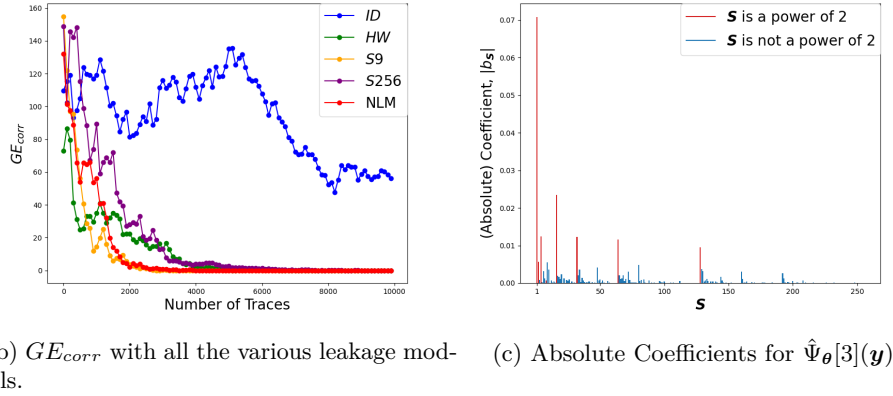


Figure 9: GE Result for ASCADf (interval of 100 traces) and absolute coefficients for closed-form expression of the NLM at the component with the highest correlation on ASCADf (Component 3).

We now aim to understand the underlying leakage captured by the NLM. To this end, we apply Algorithm 1. As in earlier instances, we display only the ten coefficients of

$\hat{\Psi}_{\theta}[3](\mathbf{y})$ with the highest absolute values:

$$\begin{aligned} \hat{\Psi}_{\theta}[3](\mathbf{y}) = & 0.0708116 * \phi_1(\mathbf{y}) + 0.00567169 * \phi_2(\mathbf{y}) + 0.01247524 * \phi_4(\mathbf{y}) \\ & + 0.00555604 * \phi_9(\mathbf{y}) + 0.02346016 * \phi_{16}(\mathbf{y}) + 0.01227226 * \phi_{32}(\mathbf{y}) \\ & - 0.00417728 * \phi_{48}(\mathbf{y}) + 0.01156934 * \phi_{64}(\mathbf{y}) + 0.00486554 * \phi_{80}(\mathbf{y}) \\ & + 0.0095234 * \phi_{128}(\mathbf{y}) + \dots \end{aligned} \quad (24)$$

As the NLM's third component provides the highest correlation with respect to the secret key, the absolute values of all the coefficients for $\hat{\Psi}_{\theta}[3](\mathbf{y})$ are plotted in Figure 9c. The observed leakage closely resembles that of AES_HD, with the largest weights corresponding to powers of 2. As before, this suggests that the underlying leakage is similar to the Hamming weight model described in Equation 19. However, for $\mathbf{S} = (0, 0, 0, 0, 1, 0, 0, 0)^3$, the corresponding absolute value of the coefficient $b_{\mathbf{S}}$ is very small, indicating that the bit $Y_k[4]$ is likely not part of the underlying leakage. Although the NLM also captures some additional components with relatively low coefficients, these extra terms may contribute to its improved performance compared to the HW. With respect to S_9 , we obtain a similar observation as in AES_HD. Firstly, the sample point 3 also has the highest correlation for the correct key when using S_9 as the leakage model. As before in AES_HD, we obtained the following equation for S_9 :

$$\begin{aligned} \hat{\Psi}_{S_9}[3](\mathbf{y}) = & 0.638712353 + 0.084081554 * \phi_1(\mathbf{y}) + 0.00324452 * \phi_2(\mathbf{y}) \\ & + 0.017268997 * \phi_4(\mathbf{y}) + 0.001570363 * \phi_8(\mathbf{y}) + 0.0246912042 * \phi_{16}(\mathbf{y}) \\ & + 0.0149532198 * \phi_{32}(\mathbf{y}) + 0.0147211272 * \phi_{64}(\mathbf{y}) + 0.00619181228 * \phi_{128}(\mathbf{y}). \end{aligned} \quad (25)$$

These results suggest that the NLM learns a leakage model closely aligned with S_9 . Overall, our experiment on ASCADf implies that our NLM remains effective even in higher-order CPA settings.

5.4.2 Experimental Results for ASCADr

Similarly, we run CPA with all the different leakage model considered and plot it in Figure 10. We further compute the GE_{corr} against the number of attack traces in an interval of 100. This is illustrated in Figure 11b with the $NTGE$ summarized in Table 11a. We observe that for ASCADr, all the leakage models are able to recover the secret key, with the HW leakage model being the best, followed by NLM and S_9 having the same $NTGE = 2500$. This shows that NLM attains comparable results even when trained from a random key setting.

As before, we compute the coefficient of the closed-form expression of NLM as stated in Section 4.2. Then, presented the closed-formed expression of NLM for the component with the highest correlation with respect to the correct key, namely, $\hat{\Psi}_{\theta}[264](\mathbf{y})$. As in previous cases, we restrict our presentation to the ten largest absolute coefficients of $\hat{\Psi}_{\theta}[264](\mathbf{y})$, and the expression is as follows,

$$\begin{aligned} \hat{\Psi}_{\theta}[264](\mathbf{y}) = & -0.01374254 + 0.02787083 * \phi_1(\mathbf{y}) + 0.04503196 * \phi_2(\mathbf{y}) \\ & + 0.03986876 * \phi_4(\mathbf{y}) + 0.04955016 * \phi_8(\mathbf{y}) + 0.03011729 * \phi_{16}(\mathbf{y}) \\ & + 0.0490356 * \phi_{32}(\mathbf{y}) + 0.02758534 * \phi_{64}(\mathbf{y}) + 0.04704924 * \phi_{128}(\mathbf{y}) \\ & + 0.00738215 * \phi_{160}(\mathbf{y}) + \dots \end{aligned} \quad (26)$$

Next, we present the absolute values $|b_{\mathbf{S}}|$ of $\hat{\Psi}_{\theta}[264](\mathbf{y})$ in Figure 11c. We observe that this expression closely resembles the Hamming weight leakage model described in Equation 19, with all the $\mathbf{S}$ that are power of 2 having the largest absolute coefficient value $|b_{\mathbf{S}}|$, i.e.

³In terms of the finite field $\mathbb{F}_2^8$, the value of S is 8, i.e. $\mathbf{S} = 8$

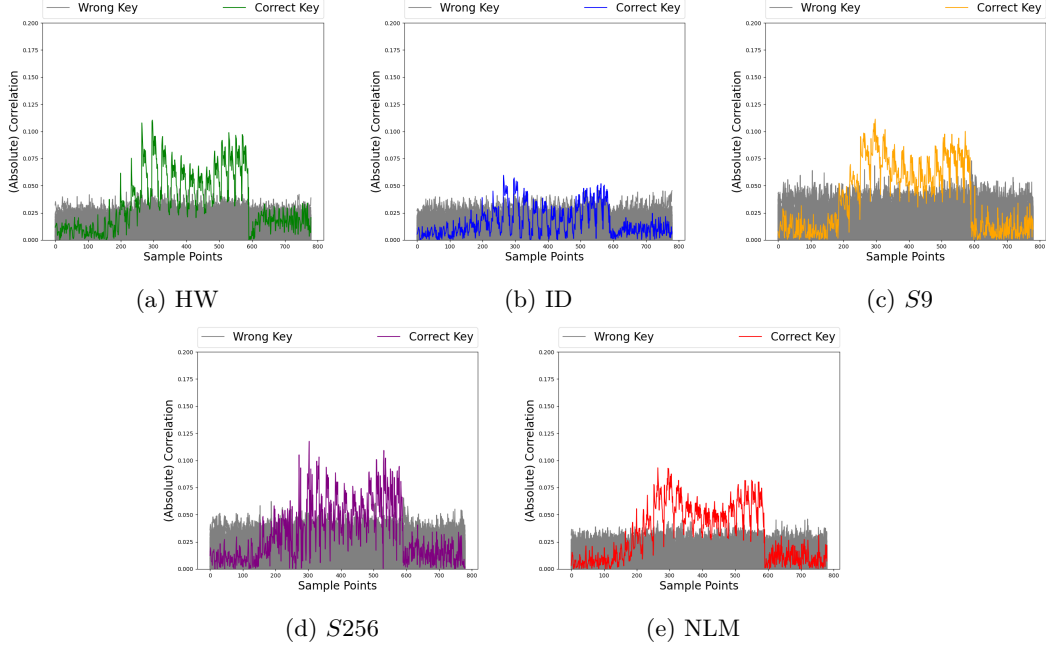


Figure 10: CPA on ASCADr datasets with classical leakage models and NLM

	ID	HW	S9	S256	NLM
<i>NTGE</i>	8400	1600	2500	2600	2500

(a) NTGE for various leakage models.

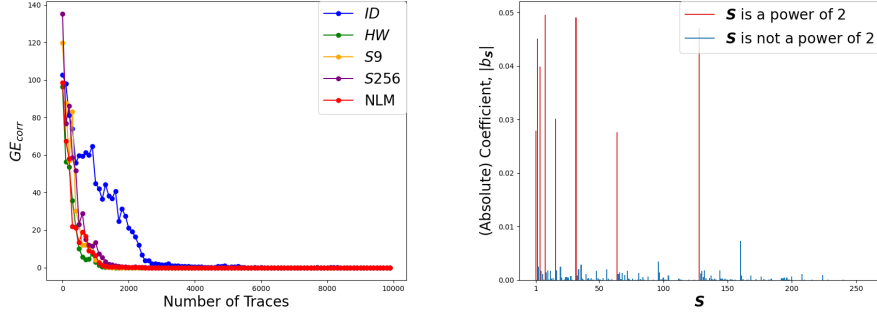
(b) GE_{corr} with all the various leakage models. (c) Absolute Coefficients for $\hat{\Psi}_{\theta}[264](y)$

Figure 11: Result with all the various leakage models for ASCADr (interval of 100 traces) and absolute coefficients for closed-form expression of the NLM at the component with the highest correlation on ASCADr (Component 264).

$S = 1, 2, 4, 8, 16, 32, 64, 128$. This is to be expected as the Hamming weight leakage model obtains the best result in terms of *NTGE* for $GE_{corr} = 0$. Hence, NLM learn a leakage model that is similar to the Hamming weight leakage model. As a result, we can conclude that NLM is comparable to classical leakage models despite being profiled with random keys.

6 Discussion

As we have seen, NLM can outperform classical leakage models in a noisier dataset like CHES2025. Why is this the case? NLMs have the capability to map the leakage into a complex, nonlinear subspace, enabling them to capture intricate and high-order interactions between input bits that characterize leakage. A natural question then arises: why does *S256*, despite its capability to model nonlinear functions, fail to recover the secret key for CHES2025, and NLM achieves better performance than *S256* across three out of the four datasets? Stochastic leakage models, such as *S256*, typically focus on the monomial basis, which was suggested by [GHTR17] to be an unsuitable representation for modeling leakage. Since the monomial basis is not orthonormal, some of its basis vectors are correlated with one another. As a result, these stochastic models produce erroneous results when computing the correlation with the underlying leakage (see Appendix A2 of [GHTR17]). Therefore, NLM provides more accurate and expressive modeling of leakage, capturing complex nonlinear interactions that traditional stochastic models often fail to detect. This theoretical argument could be the reason the NLM succeeded in recovering the secret key on the CHES2025 dataset, whereas classical leakage models failed.

Beyond improved performance, the closed-form expression of NLM offers additional benefits: it allows evaluators to gain insight into the interactions between bits, potentially guiding designers to modify the underlying implementation. However, since this study focuses primarily on public datasets, exploring implementation changes is beyond its scope and is left for future work.

Despite these advantages, NLM is not without limitations. As NLM produces outputs for each individual sample point, its performance may degrade when processing excessively long traces, since it becomes computationally more expensive and takes longer to train. A possible approach to mitigate this issue is to partition the traces into smaller segments and train a separate NLM for each segment. The question of how to optimally determine the segment length, however, is non-trivial and is left as a topic for future investigation.

7 Conclusion

In this work, we proposed a new type of profiled leakage model based on neural networks, referred to as NLM. The NLM takes a sensitive variable as input and learns the deterministic part of the leakage through training. Additionally, we introduced an algorithm based on Fourier expansion to convert the trained NLM into a polynomial representation with respect to the Fourier basis. This closed-form expression is based on the Fourier basis where the basis vectors are uncorrelated. This provides a clear interpretation of the NLM through its exact expression.

We assessed our approach on four datasets, covering both hardware and software implementations. First, we conducted experiments on AES_HD, a hardware implementation, and showed that the NLM achieves performance comparable to the classical leakage model. We then evaluated our NLM on the CHES2025 dataset, which has traces acquired that are highly desynchronized. Notably, the NLM was the only leakage model capable of successfully recovering the secret key in this challenging setting, where all other classical leakage models failed. As far as we are aware, this is the first work to precisely identify the locations of the leakages in the CHES2025 dataset, made possible through the application of the NLM. Finally, we tested our method on two masked datasets: ASCADf (fixed key) and ASCADr (random key). In both cases, the NLM successfully enabled key recovery using CPA, achieving performance on par with classical leakage models. A natural direction for further investigation is to explore how the choice of combining functions in higher-order CPA influences the leakage model learned by the NLM, and whether this combining function can be incorporated directly into the model architecture. Another

possible direction would be to explore alternative loss functions that could better handle masking.

References

- [AARR02] Dakshi Agrawal, Bruce Archambeault, Josyula R. Rao, and Pankaj Rohatgi. The EM Side-Channel(s). In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*, pages 29–45. Springer, 2002.
- [ABDM00] Mehdi-Laurent Akkar, Régis Bevan, Paul Dischamps, and Didier Moyart. Power Analysis, What Is Now Possible... In Tatsuaki Okamoto, editor, *Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings*, volume 1976 of *Lecture Notes in Computer Science*, pages 489–502. Springer, 2000.
- [BBJ⁺25] Shivam Bhasin, Harishma Boyapally, Dirmanto Jap, Trevor Yap, and Qianmei Wu. GE Wars: The Deep Learning SCA battle. CHES Challenge 2025, 2025. <https://pace-tl.gitbook.io/ches-challenge-2025>.
- [BCG⁺25] Sana Boussam, Mathieu Carbone, Benoît Gérard, Guénaél Renault, and Gabriel Zaid. Optimal Dimensionality Reduction using Conditional Variational AutoEncoder. *IACR Cryptol. ePrint Arch.*, page 1134, 2025.
- [BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation Power Analysis with a Leakage Model. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings*, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004.
- [Bis07] Christopher M. Bishop. *Pattern recognition and machine learning, 5th Edition*. Information science and statistics. Springer, 2007.
- [BJW⁺25] Harishma Boyapally, Dirmanto Jap, Qianmei Wu, Fan Zhang, and Shivam Bhasin. Reality check on side-channels: lessons learnt from breaking AES on ARM Cortex-A72 processor with out-of-order execution. In *Proceedings of the 19th USENIX Conference on Offensive Technologies*, WOOT '25, USA, 2025. USENIX Association.
- [BPS⁺20] Ryad Benadjila, Emmanuel Prouff, Rémi Strullu, Eleonora Cagli, and Cécile Dumas. Deep learning for side-channel analysis and introduction to ASCAD database. *J. Cryptogr. Eng.*, 10(2):163–188, 2020.
- [Car20] Claude Carlet, editor. *Boolean Functions for Cryptography and Coding Theory*. Cambridge University Press, 2020.
- [CDP17] Eleonora Cagli, Cécile Dumas, and Emmanuel Prouff. Convolutional Neural Networks with Data Augmentation Against Jitter-Based Countermeasures - Profiling Attacks Without Pre-processing. In Wieland Fischer and Naofumi Homma, editors, *Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, volume 10529 of *Lecture Notes in Computer Science*, pages 45–68. Springer, 2017.

- [CK14] Marios O. Choudary and Markus G. Kuhn. Efficient Stochastic Methods: Profiled Attacks Beyond 8 Bits. In Marc Joye and Amir Moradi, editors, *Smart Card Research and Advanced Applications - 13th International Conference, CARDIS 2014, Paris, France, November 5-7, 2014. Revised Selected Papers*, volume 8968 of *Lecture Notes in Computer Science*, pages 85–103. Springer, 2014.
- [CNK⁺22] Juncheng Chen, Jun-Sheng Ng, Nay Aung Kyaw, Ne Kyaw Zwa Lwin, Kwen-Siong Chong, Zhiping Lin, Joseph Sylvester Chang, and Bah-Hwee Gwee. Non-profiling based Correlation Optimization Deep Learning Analysis. In *IEEE International Symposium on Circuits and Systems, ISCAS 2022, Austin, TX, USA, May 27 - June 1, 2022*, pages 2246–2250. IEEE, 2022.
- [GBC16] Ian J. Goodfellow, Yoshua Bengio, and Aaron C. Courville. *Deep Learning*. Adaptive computation and machine learning. MIT Press, 2016.
- [GHTR17] Sylvain Guilley, Annelie Heuser, Ming Tang, and Olivier Rioul. Stochastic Side-Channel Leakage Analysis via Orthonormal Decomposition. In Pooya Farshim and Emil Simion, editors, *Innovative Security Solutions for Information Technology and Communications - 10th International Conference, SecITC 2017, Bucharest, Romania, June 8-9, 2017, Revised Selected Papers*, volume 10543 of *Lecture Notes in Computer Science*, pages 12–27. Springer, 2017.
- [GLP06] Benedikt Gierlichs, Kerstin Lemke-Rust, and Christof Paar. Templates vs. Stochastic Methods. In Louis Goubin and Mitsuru Matsui, editors, *Cryptographic Hardware and Embedded Systems - CHES 2006, 8th International Workshop, Yokohama, Japan, October 10-13, 2006, Proceedings*, volume 4249 of *Lecture Notes in Computer Science*, pages 15–29. Springer, 2006.
- [HKSS12] Annelie Heuser, Michael Kasper, Werner Schindler, and Marc Stöttinger. A New Difference Method for Side-Channel Analysis with High-Dimensional Leakage Models. In Orr Dunkelman, editor, *Topics in Cryptology - CT-RSA 2012 - The Cryptographers’ Track at the RSA Conference 2012, San Francisco, CA, USA, February 27 - March 2, 2012. Proceedings*, volume 7178 of *Lecture Notes in Computer Science*, pages 365–382. Springer, 2012.
- [HLF⁺21] Benjamin Hettwer, Sebastien Leger, Daniel Fennes, Stefan Gehrler, and Tim Güneysu. Side-Channel Analysis of the Xilinx Zynq UltraScale+ Encryption Engine. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(1):279–304, 2021.
- [JSB15] Dirmanto Jap, Marc Stöttinger, and Shivam Bhasin. Support vector regression: exploiting machine learning techniques for leakage modeling. In Ruby B. Lee, Weidong Shi, and Jakub Szefer, editors, *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy, HASP@ISCA 2015, Portland, OR, USA, June 14, 2015*, pages 2:1–2:8. ACM, 2015.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential Power Analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO ’96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.

- [KSS10] Michael Kasper, Werner Schindler, and Marc Stöttinger. A stochastic method for security evaluation of cryptographic FPGA implementations. In Jinian Bian, Qiang Zhou, Peter Athanas, Yajun Ha, and Kang Zhao, editors, *Proceedings of the International Conference on Field-Programmable Technology, FPT 2010, 8-10 December 2010, Tsinghua University, Beijing, China*, pages 146–153. IEEE, 2010.
- [Lue21] Knud Lasse Lueth. State of the IoT 2020: 12 billion IoT connections, surpassing non-IoT for the first time, 2021.
- [MPP16] Housseem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. Breaking Cryptographic Implementations Using Deep Learning Techniques. In Claude Carlet, M. Anwar Hasan, and Vishal Saraswat, editors, *Security, Privacy, and Applied Cryptography Engineering - 6th International Conference, SPACE 2016, Hyderabad, India, December 14-18, 2016, Proceedings*, volume 10076 of *Lecture Notes in Computer Science*, pages 3–26. Springer, 2016.
- [O’D21] Ryan O’Donnell. Analysis of Boolean Functions. *CoRR*, abs/2105.10386, 2021.
- [PR13] Emmanuel Prouff and Matthieu Rivain. Masking against Side-Channel Attacks: A Formal Security Proof. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings*, volume 7881 of *Lecture Notes in Computer Science*, pages 142–159. Springer, 2013.
- [PRB10] Emmanuel Prouff, Matthieu Rivain, and Régis Bevan. Statistical analysis of second order differential power analysis. *IACR Cryptol. ePrint Arch.*, page 646, 2010.
- [RQL19] Pieter Robyns, Peter Quax, and Wim Lamotte. Improving CEMA using Correlation Optimization. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(1):1–24, 2019.
- [RYJ⁺25] Azade Rezaeezade, Trevor Yap, Dirmanto Jap, Shivam Bhasin, and Stjepan Picek. Breaking the Blindfold: Deep Learning-based Blind Side-channel Analysis. *IACR Cryptol. ePrint Arch.*, page 157, 2025.
- [SET23] Thomas Schamberger, Maximilian Egger, and Lars Tebelmann. Hide and Seek: Using Occlusion Techniques for Side-Channel Leakage Attribution in CNNs - An Evaluation of the ASCAD Databases. In *Applied Cryptography and Network Security Workshops - ACNS 2023 Satellite Workshops, ADSC, AIBlock, AIHWS, AIoTS, CIMSS, Cloud S&P, SCI, SecMT, SiMLA, Kyoto, Japan, June 19-22, 2023, Proceedings*, volume 13907 of *Lecture Notes in Computer Science*, pages 139–158. Springer, 2023.
- [SKP⁺24] Ioana Savu, Marina Krcek, Guilherme Perin, Lichao Wu, and Stjepan Picek. The Need for MORE: Unsupervised Side-Channel Analysis with Single Network Training and Multi-output Regression. In Romain Wacquez and Naofumi Homma, editors, *Constructive Side-Channel Analysis and Secure Design - 15th International Workshop, COSADE 2024, Gardanne, France, April 9-10, 2024, Proceedings*, volume 14595 of *Lecture Notes in Computer Science*, pages 113–132. Springer, 2024.
- [SLP05] Werner Schindler, Kerstin Lemke, and Christof Paar. A Stochastic Model for Differential Side Channel Cryptanalysis. In Josyula R. Rao and Berk

- Sunar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2005, 7th International Workshop, Edinburgh, UK, August 29 - September 1, 2005, Proceedings*, volume 3659 of *Lecture Notes in Computer Science*, pages 30–46. Springer, 2005.
- [SM23] Marvin Staib and Amir Moradi. Deep Learning Side-Channel Collision Attack. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(3):422–444, 2023.
- [SPQ05] François-Xavier Standaert, Eric Peeters, and Jean-Jacques Quisquater. On the Masking Countermeasure and Higher-Order Power Analysis Attacks. In *International Symposium on Information Technology: Coding and Computing (ITCC 2005), Volume 1, 4-6 April 2005, Las Vegas, Nevada, USA*, pages 562–567. IEEE Computer Society, 2005.
- [Tim19] Benjamin Timon. Non-Profiled Deep Learning-based Side-Channel attacks with Sensitivity Analysis. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(2):107–131, 2019.
- [WCL⁺20] Ping Wang, Ping Chen, Zhimin Luo, Gaofeng Dong, Mengce Zheng, Nenghai Yu, and Honggang Hu. Enhancing the Performance of Practical Profiling Side-Channel Attacks Using Conditional Generative Adversarial Networks. *CoRR*, abs/2007.05285, 2020.
- [WRA⁺24] Lichao Wu, Azade Rezaeezade, Amir Alipour, Guilherme Perin, and Stjepan Picek. Leakage Model-flexible Deep Learning-based Side-channel Analysis. *IACR Commun. Cryptol.*, 1(3):41, 2024.
- [WTPP24] Lichao Wu, Sébastien Tiran, Guilherme Perin, and Stjepan Picek. Plaintext-based side-channel collision attack. *IACR Commun. Cryptol.*, 1(3):20, 2024.
- [WW04] Jason Waddle and David A. Wagner. Towards Efficient Second-Order Power Analysis. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings*, volume 3156 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2004.
- [YJ24] Trevor Yap and Dirmanto Jap. Creating from noise: Trace generations using diffusion model for side-channel attack. In Martin Andreoni, editor, *Applied Cryptography and Network Security Workshops - ACNS 2024 Satellite Workshops, AIBlock, AIHWS, AIoTS, SCI, AAC, SiMLA, LLE, and CIMSS, Abu Dhabi, United Arab Emirates, March 5-8, 2024, Proceedings, Part I*, volume 14586 of *Lecture Notes in Computer Science*, pages 102–120. Springer, 2024.
- [YPB24] Trevor Yap, Stjepan Picek, and Shivam Bhasin. Occpois: Points of interest based on neural network’s key recovery in side-channel analysis through occlusion. In Sourav Mukhopadhyay and Pantelimon Stanica, editors, *Progress in Cryptology - INDOCRYPT 2024 - 25th International Conference on Cryptology in India, Chennai, India, December 18-21, 2024, Proceedings, Part II*, volume 15496 of *Lecture Notes in Computer Science*, pages 3–28. Springer, 2024.
- [ZBC⁺23] Gabriel Zaid, Lilian Bossuet, Mathieu Carbone, Amaury Habrard, and Alexandre Venelli. Conditional Variational AutoEncoder based on Stochastic Attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):310–357, 2023.